

Repaso conceptos básicos de la API

Una API (interfaz de programación de aplicaciones) permite que dos sistemas informáticos interactúen entre sí. Por ejemplo, si creamos una automatización que genera un informe y lo envía por email, el envío de ese email no se hace manualmente, lo hará el propio script. Para ello, Python (o el lenguaje que utilicemos), debe pedir a Gmail que envíe este correo electrónico, con ese informe adjunto a determinadas personas. La forma de hacerlo es a través de una API, en este caso la API de Gmail.

Recordemos cuáles son las partes principales de una API:

- **Protocolo de transferencia HTTP** : es la principal forma de comunicar información en la web. Existen diferentes métodos, cada uno de ellos utilizado para diferentes problemas:
 - **GET** : este método permite obtener información de la base de datos o de un proceso.
 - **POST** : permite enviar información, ya sea para agregar información a una base de datos o para pasar la entrada de un modelo de aprendizaje automático, por ejemplo.
 - **PUT**: actualizar información. Generalmente se utiliza para gestionar información en la base de datos.
 - **DELETE** : este método se utiliza para eliminar información de la base de datos.
- **Url** : es la dirección donde podemos encontrar nuestra API. Básicamente esta URL constará de tres partes:
 - **Protocolo** : puede ser `http://`, `https://`, `ftp://`, etc
 - **Dominio** : el host en el que está alojado, que va desde el protocolo hasta el final de `.com`, o el final que tenga la url. Por ejemplo, el dominio del campus es `campusv.fceia.unr.edu.ar`.
 - **Puerto**: por defecto es el puerto 80 para http, o el 443 para https, si no, se aclara luego de `:`, por ejemplo habrán visto el `:8000` que usa jupyter
 - **Terminación**: como un sitio web tiene varias páginas (`/blog`), (`/legal`), una misma API puede incluir múltiples puntos y cada uno hace cosas diferentes. Al crear nuestra API en Python, indicaremos las terminaciones, por lo que debemos asegurarnos de que cada una sea representativa de lo que hace la API detrás de él.

Cómo crear una API en Python con FastAPI

Se podría crear una API desde cero, pero existen diferentes marcos de trabajo (frameworks) que nos ayudan con esta tarea, una de las más utilizadas ultimamente es FastAPI. Existen otras alternativas para python como Flask, Django REST Framework (DRF), y luego muchas más para otros lenguajes de programación. Nosotros usaremos FastAPI, aunque deben saber que no es la única opción.

Cómo crear una API en Python con FastAPI

Requisitos para usar FastAPI

Para usar FastAPI, se debe tener Python 3.6 en adelante e instalar dos bibliotecas:

`fastapi` y `uvicorn`.

```
pip install fastapi
```

```
pip install uvicorn
```

Tu primera API de Python con FastAPI

Ahora que hemos instalado los paquetes, simplemente debemos crear un archivo en Python donde definiremos nuestra API. En este archivo debemos crear una app, donde incluiremos las APIs, con sus terminaciones, parámetros, etc.

Una vez que tenemos la aplicación, ahí es donde definimos la información que requiere la API: punto final, método HTTP, argumentos de entrada y qué hará la API detrás de esto.

```
from fastapi import FastAPI
app = FastAPI()
```

```
@app.get("/my-first-api")
def hello():
    return {"Hello world!"}
```

Con esto ya tendríamos una API muy sencilla creada, que simplemente devuelve “¡Hola mundo!”. Como verás, en unas pocas líneas hemos definido: el método (get), la terminación (“/”) y la función que debe ejecutar esta API.

Incluso podríamos pasar argumentos a nuestra API, para que los use en su función. Siempre que pasemos un argumento a nuestra función debemos indicar el tipo de dato que debe ser (número, texto, etc.).

Importante : FastAPI realiza una comprobación de que el tipo de datos que le pasamos en la llamada es el que le hemos indicado que debe ser. Esto es esencial para garantizar que nuestra API funcione correctamente, y es algo que otros marcos de creación de API (como Flask) no incluyen.

Veamos cómo funciona en un ejemplo:

```

from fastapi import FastAPI
app = FastAPI()

@app.get("/my-first-api")
def hello(name: str):
    return {'Hello ' + name + '!'}

```

Ahora, cuando hagamos la solicitud a esta API, tendremos que pasarle el parámetro de nombre para que funcione. Es decir, si antes nos bastaba con ir a

<http://127.0.0.1:8000/my-first-api>, ahora tendremos que pasar el parámetro `name`. Así la solicitud quedará de la siguiente manera:
<http://127.0.0.1:8000/my-first-api?name=MNombre> .

Como hemos incluido el argumento `name`, este argumento es obligatorio: si no se incluye en la solicitud, no funcionará. Sin embargo, es posible que queramos pasar argumentos opcionales.

Para ello, tenemos que indicar el argumento como `None` y FastAPI lo interpretará correctamente. Ejemplo: vamos a crear una API a la que le puedes pasar, o no, la variable `name`. Si lo pasa, devolverá “¡Hola {nombre}!” y, si no, simplemente “¡Hola!”.

```

from fastapi import FastAPI

app = FastAPI()

@app.get("/my-first-api")
def hello(name = None):

    if name is None:
        text = 'Hello!'

    else:
        text = 'Hello ' + name + '!'

    return text

```

En este caso, si vamos a <http://127.0.0.1:8000/my-first-api> la API se ejecutará correctamente y devolverá “¡Hola!”, mientras que si le pasamos el parámetro, por ejemplo, <http://127.0.0.1:8000/my-first-api?name=MNombre> lo utilizará y seguirá funcionando correctamente, devolviendo `Hello MNombre!`.

Devolver diferentes tipos de datos con FastAPI

La gran mayoría de veces una API suele devolver texto (una predicción, datos, etc.), aunque muchas veces puede devolver otro tipo de datos, como un DataFrame o una imagen, por ejemplo.

Cuando se trata de objetos "normales", como un DataFrame, FastAPI lo convertirá directamente en un archivo JSON. Ejemplo:

```
from fastapi import FastAPI

app = FastAPI()

@app.get("/get-iris")
def get_iris():

    import pandas as pd
    url
    ='https://gist.githubusercontent.com/curran/a08a1080b88344b0c8a7/raw/0e7a9b0a5d22642a06d3d5b9bcbad9890c8ee534/iris.csv'
    iris = pd.read_csv(url)

    return iris
```

Si hacemos una petición a esta terminación, recibiremos la siguiente respuesta:

```
resp.text
'{"sepal_length":{"0":5.1,"1":4.9,"2":4.7,"3":4.6,"4":5.0,"5":5.4,"6":4.6,
"7":5.0,"8":4.4,"9":4.9,"10":5.4,"11":4.8,"12":4.8,"13":4.3,"14":5.8,"15":
5.7,"16":5.4,"17":5.1,"18":5.7,"19":5.1,"20":5.4,"21":5.1,"22":4.6,"23":5.
1,"24":4.8,"25":5.0,"26":5.0,"27":5.2,"28":5.2,"29":4.7,"30":4.8,"31":5.4,
"32":5.2,"33":5.5,"34":4.9,"35":5.0,"36":5.5,"37":4.9,"38":4.4,"39":5.1,"4
0":5.0,"41":4.5,"42":4.4,"43":5.0,"44":5.1,"45":4.8,"46":5.1,"47":4.6,"48"
:5.3,"49":5.0,"50":7.0,...
```

Como puede ver, Fast API convierte el marco de datos directamente en un objeto JSON. Sin embargo, ¿qué pasa con las imágenes o videos?

FastAPI se basa en [starlette](#), por lo que para responder con imágenes o íconos, podemos usar ambos [StreamingResponse](#) y [FileResponse](#). En cualquier caso, necesitaremos instalar [aiofiles](#).

Entonces, si queremos mostrar una imagen en nuestra API, nuestra aplicación FastAPI se verá así:

```

@app.get("/plot-iris")
def plot_iris():

    import pandas as pd
    import matplotlib.pyplot as plt

    url
    ='https://gist.githubusercontent.com/curran/a08a1080b88344b0c8a7/raw/0e7a9
    b0a5d22642a06d3d5b9bcbad9890c8ee534/iris.csv'
    iris = pd.read_csv(url)

    plt.scatter(iris['sepal_length'], iris['sepal_width'])
    plt.savefig('iris.png')
    file = open('iris.png', mode="rb")

    return StreamingResponse(file, media_type="image/png")

```

Comprobar el funcionamiento de una API FastAPI

Como decía al principio de este apartado, para crear una API en FastAPI debemos incluir nuestro código en un archivo de Python, preferiblemente `main.py`. Además, debemos tener instalado `uvicorn`. Teniendo esto en cuenta, podemos ejecutar nuestra API de una forma muy sencilla, con el siguiente código:

```
uvicorn main:app --reload
```

Esto ejecutará nuestra aplicación y podremos acceder a nuestra API, tanto a nivel de navegador como realizando llamadas desde nuestra computadora. La API aún no se ha cargado en ningún lugar, por lo que se podrá acceder desde localhost. Puedes acceder a localhost tanto en `http://127.0.0.1/` como en `http://localhost/`, aunque tendrás que hacerlo en el mismo puerto en el que se está ejecutando la API. En general es el puerto 8000, pero te lo indica el mismo uvicorn al ejecutarlo

```

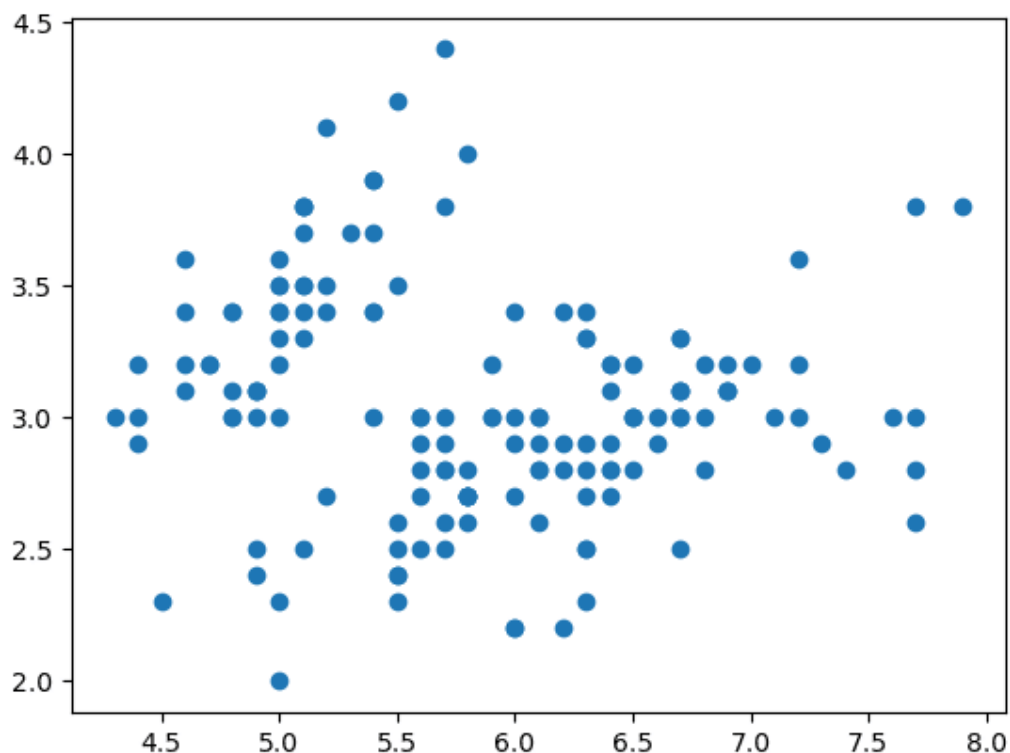
$uvicorn main:app --reload
INFO:     Will watch for changes in these directories: ['/home/informatica/facu/redes/api']
INFO:     Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
INFO:     Started reloader process [11824] using StatReload
INFO:     Started server process [11826]
INFO:     Waiting for application startup.
INFO:     Application startup complete.

```

Veamos por ejemplo cómo le pido a una API que incluya todas las terminaciones que hicimos hasta ahora:

```
import requests
from PIL import Image
import io

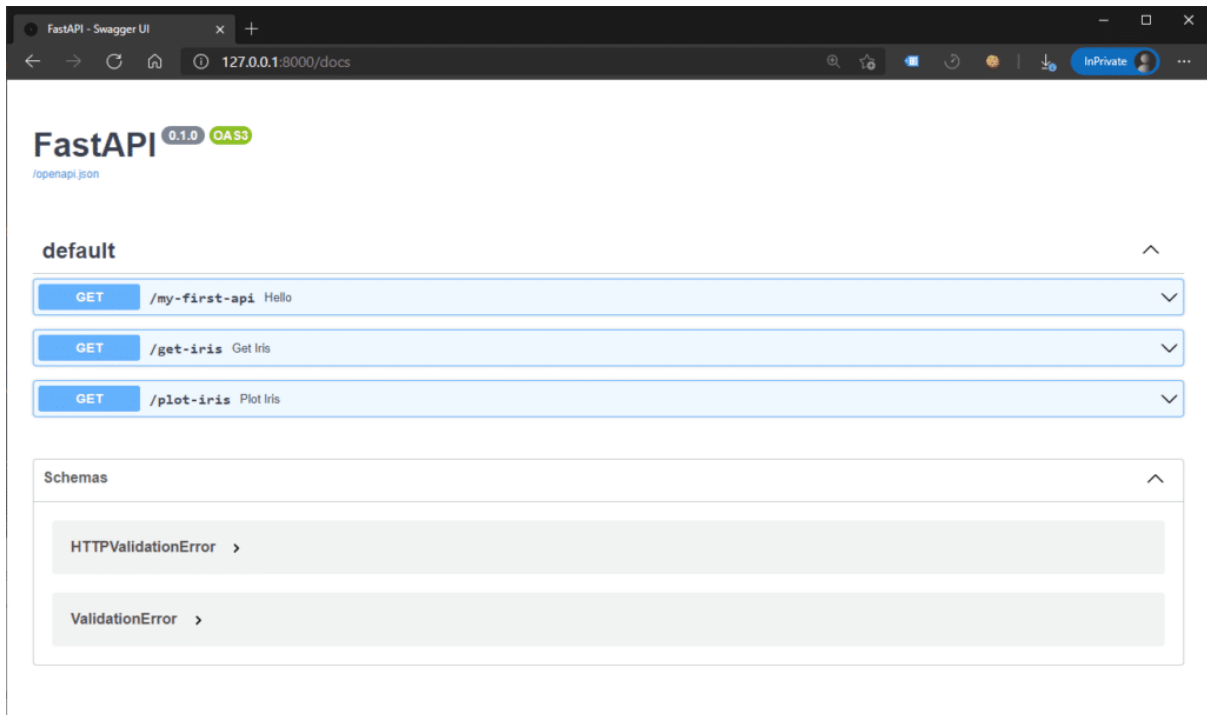
resp = requests.get('http://127.0.0.1:8000/plot-iris')
file = io.BytesIO(resp.content)
im = Image.open(file)
im.show()
```



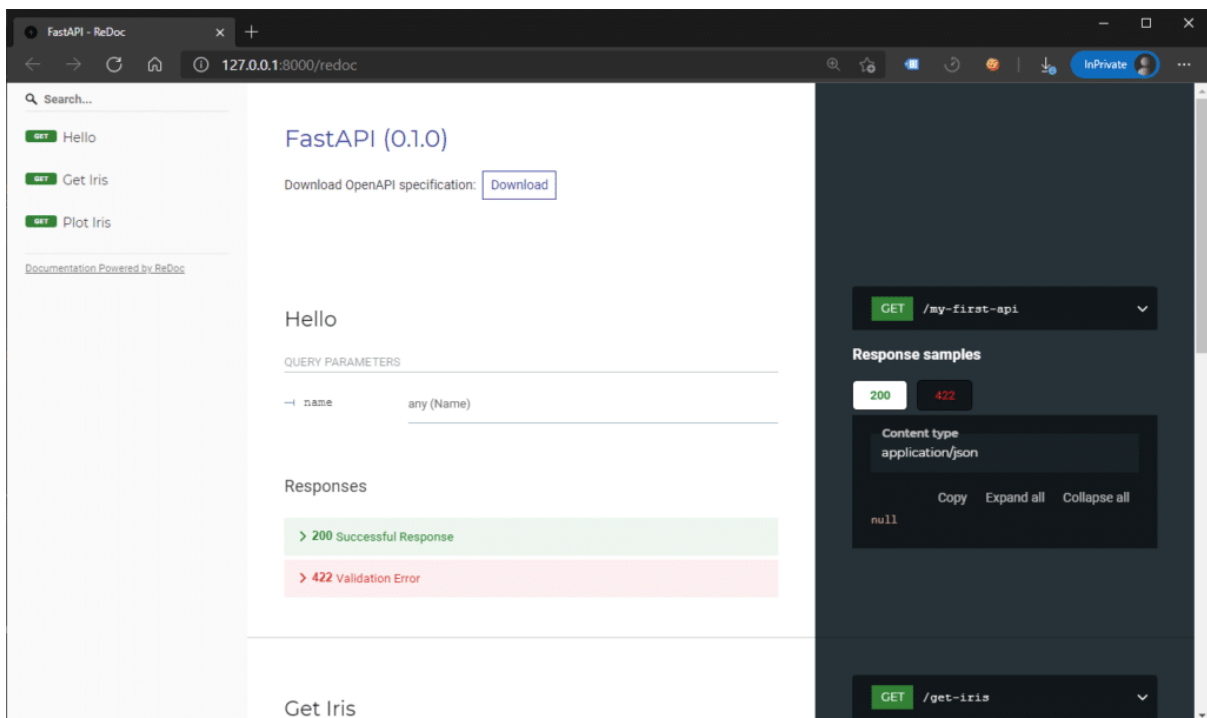
Como vemos nuestra API devuelve la imagen.
 Probemos ahora la terminación my-fist-api:

```
resp = requests.get('http://127.0.0.1:8000/my-first-api?name=Andrea')
resp.text
'"Hello Andrea!"'
```

FastAPI cuando lo usamos nos crea “gratis” la documntaciòn en swagger, un motor de documentaciòn de python, podemos accederlo en la terminaciòn docs. En mi caso puedo acceder como <http://127.0.0.1:8000/docs>, como vemos en la siguiente imagen:



Ademas, hay otra forma de documentar la API, con OpenAPI, que se puede acceder a través de la terminación `/redoc`, como vemos acá:



Fuente:

anderfernandez.com/en/blog/how-to-create-api-python/#How-to-create-an-API-in-Python